

**CENTRO DE EDUCAÇÃO TECNOLÓGICA DO AMAZONAS**

*Qualificação Profissional em Lógica de Programação Básica*

**APOSTILA DE**  
**LÓGICA DE PROGRAMAÇÃO**  
**com Portugol e VisuAlg**

*Material didático para alunos iniciantes em programação*

**Professora: Esp. Priscila Gonçalves**

Manaus – Amazonas

# Sumário

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| <b>Capítulo 1 — Introdução à Lógica de Programação.....</b>      | <b>4</b>  |
| 1.1 O que é lógica de programação? .....                         | 4         |
| 1.2 Raciocínio lógico no dia a dia .....                         | 4         |
| 1.3 Abstração e resolução de problemas.....                      | 4         |
| 1.4 Exercícios — Capítulo 1.....                                 | 5         |
| <b>Capítulo 2 — Algoritmos e Fluxogramas .....</b>               | <b>6</b>  |
| 2.1 O que é um algoritmo?.....                                   | 6         |
| Características de um bom algoritmo.....                         | 6         |
| 2.2 Formas de representar um algoritmo.....                      | 6         |
| 2.3 Símbolos básicos de fluxograma.....                          | 6         |
| 2.4 Exemplo — do problema ao algoritmo.....                      | 7         |
| 2.5 Exercícios — Capítulo 2.....                                 | 7         |
| <b>Capítulo 3 — Conhecendo o VisuAlg e o Portugol.....</b>       | <b>9</b>  |
| 3.1 O que é o Portugol? .....                                    | 9         |
| 3.2 O que é o VisuAlg? .....                                     | 9         |
| 3.3 Estrutura básica de um algoritmo em VisuAlg .....            | 9         |
| Explicando cada parte.....                                       | 9         |
| 3.4 Primeiro programa: "Olá, Mundo!" .....                       | 9         |
| 3.5 Regras gerais de sintaxe.....                                | 10        |
| 3.6 Exercícios — Capítulo 3.....                                 | 10        |
| <b>Capítulo 4 — Variáveis, Constantes e Tipos de Dados .....</b> | <b>11</b> |
| 4.1 O que são variáveis? .....                                   | 11        |
| 4.2 Regras para nomear variáveis.....                            | 11        |
| 4.3 Tipos de dados primitivos no VisuAlg .....                   | 11        |
| 4.4 Declarando variáveis .....                                   | 11        |
| 4.5 Constantes .....                                             | 11        |
| 4.6 Atribuição de valores.....                                   | 12        |
| 4.7 Exercícios — Capítulo 4.....                                 | 12        |
| <b>Capítulo 5 — Operadores e Expressões .....</b>                | <b>13</b> |
| 5.1 Operadores aritméticos.....                                  | 13        |
| 5.2 Operadores relacionais.....                                  | 13        |
| 5.3 Operadores lógicos .....                                     | 13        |
| 5.4 Ordem de precedência dos operadores.....                     | 13        |
| 5.5 Exemplo prático .....                                        | 14        |
| 5.6 Exercícios — Capítulo 5.....                                 | 14        |
| <b>Capítulo 6 — Entrada e Saída de Dados .....</b>               | <b>15</b> |
| 6.1 Comandos de saída: escreva() e escreval() .....              | 15        |
| 6.2 Comando de entrada: leia() .....                             | 15        |
| 6.3 Concatenando textos e variáveis .....                        | 15        |

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| 6.4 Exemplo completo — Calculadora de Média .....                | 16        |
| 6.5 Exercícios — Capítulo 6.....                                 | 16        |
| <b>Capítulo 7 — Estruturas Condicionais.....</b>                 | <b>17</b> |
| 7.1 Por que usar estruturas condicionais? .....                  | 17        |
| 7.2 Seleção simples: se ... entao .....                          | 17        |
| 7.3 Seleção composta: se ... entao ... senao .....               | 17        |
| 7.4 Condições encadeadas: senao se .....                         | 18        |
| 7.5 Seleção múltipla: escolha ... caso .....                     | 18        |
| 7.6 Exercícios — Capítulo 7.....                                 | 19        |
| <b>Capítulo 8 — Estruturas de Repetição .....</b>                | <b>20</b> |
| 8.1 Por que usar estruturas de repetição? .....                  | 20        |
| 8.2 Estrutura para (contador definido) .....                     | 20        |
| 8.3 Estrutura enquanto (repetição com teste no início) .....     | 20        |
| 8.4 Estrutura repita ... ate (repetição com teste no final)..... | 21        |
| 8.5 Comparando as três estruturas .....                          | 21        |
| 8.6 Exercícios — Capítulo 8.....                                 | 22        |
| <b>Capítulo 9 — Vetores (Introdução) .....</b>                   | <b>23</b> |
| 9.1 O que é um vetor?.....                                       | 23        |
| 9.2 Declarando um vetor .....                                    | 23        |
| 9.3 Acessando e preenchendo um vetor .....                       | 23        |
| 9.4 Exercícios — Capítulo 9.....                                 | 24        |
| <b>Capítulo 10 — Projeto Prático Final .....</b>                 | <b>25</b> |
| 10.1 Objetivo do projeto.....                                    | 25        |
| 10.2 Proposta: Sistema de Cadastro de Produtos .....             | 25        |
| 10.3 Estrutura sugerida do algoritmo.....                        | 25        |
| 10.4 Desafios extras .....                                       | 26        |
| <b>Referências .....</b>                                         | <b>27</b> |

# Capítulo 1 — Introdução à Lógica de Programação

## 1.1 O que é lógica de programação?

Lógica de programação é a forma organizada de pensar que usamos para resolver problemas por meio de uma sequência de passos claros e ordenados. Antes de escrever qualquer linha de código em uma linguagem de programação, é preciso desenvolver o raciocínio lógico: a capacidade de analisar um problema, dividi-lo em partes menores e definir, passo a passo, como resolvê-lo.

Programar não é decorar comandos de uma linguagem. Programar é, antes de tudo, pensar de forma estruturada. Por isso, este material tem como ponto de partida o desenvolvimento do raciocínio lógico, para só depois apresentar a sintaxe da linguagem Portugol, utilizada no ambiente VisuAlg.

## 1.2 Raciocínio lógico no dia a dia

Sem perceber, usamos lógica em diversas situações cotidianas. Preparar uma receita, trocar um pneu, montar um móvel ou até escolher o caminho mais curto para o trabalho são tarefas que seguem uma sequência lógica de passos. Observe o exemplo a seguir:

Tarefa: Fazer um café  
1. Colocar água na chaleira  
2. Ligar a chaleira e esperar ferver  
3. Colocar pó de café no filtro  
4. Despejar a água quente sobre o pó  
5. Servir o café na xícara

Perceba que essa sequência precisa seguir uma ordem: não é possível servir o café antes de coar, nem coar antes de ferver a água. Essa mesma lógica de sequência ordenada de passos é a base de qualquer algoritmo de programação.

## 1.3 Abstração e resolução de problemas

Abstração é a habilidade de identificar, em um problema real, apenas as informações essenciais para resolvê-lo, deixando de lado detalhes que não são relevantes para a solução. Ao programar, precisamos abstrair o problema do mundo real e traduzi-lo em dados, decisões e ações que o computador consiga executar.

Para desenvolver o raciocínio lógico, é útil seguir um método simples ao resolver qualquer problema:

1. Entender o problema: o que exatamente precisa ser resolvido?
2. Identificar os dados de entrada: quais informações eu tenho ou preciso pedir ao usuário?
3. Definir o processamento: quais cálculos, comparações ou decisões são necessários?
4. Definir a saída: qual é o resultado final que deve ser apresentado?
5. Testar a solução: a lógica funciona para diferentes situações e casos?

### Dica da professora

Sempre que for resolver um exercício de lógica, escreva em português comum (sem se preocupar com comandos) o passo a passo da solução antes de tentar programar. Essa técnica é chamada de pseudocódigo livre e ajuda muito a organizar as ideias.

## **1.4 Exercícios — Capítulo 1**

1. Descreva, em uma sequência numerada de passos, como você faria para trocar uma lâmpada queimada.
2. Escreva o passo a passo lógico para atravessar uma rua com segurança.
3. Pense em uma tarefa do seu cotidiano e identifique: quais são os dados de entrada, o processamento e a saída dessa tarefa?
4. Explique, com suas palavras, a diferença entre lógica de programação e uma linguagem de programação.

## Capítulo 2 — Algoritmos e Fluxogramas

### 2.1 O que é um algoritmo?

Um algoritmo é uma sequência finita, ordenada e não ambígua de passos que leva à solução de um problema. Todo algoritmo precisa ter um início, um conjunto de instruções claras e um fim. As instruções devem ser precisas o suficiente para que qualquer pessoa (ou computador) consiga executá-las sem margem para interpretações diferentes.

#### Características de um bom algoritmo

- **Finitude:** deve ter um número limitado de passos e terminar em algum momento.
- **Clareza:** cada passo deve ser bem definido, sem ambiguidade.
- **Precisão na ordem:** a sequência dos passos deve ser respeitada.
- **Eficácia:** deve realmente resolver o problema proposto.

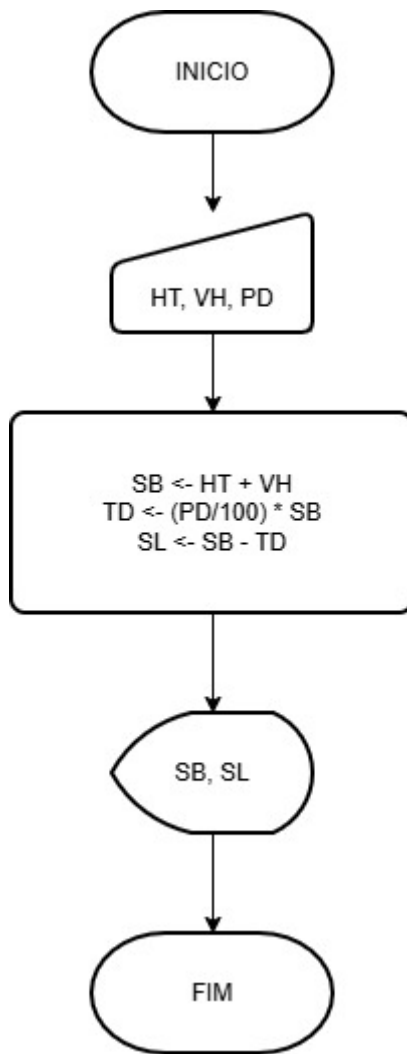
### 2.2 Formas de representar um algoritmo

Existem três formas mais comuns de representar um algoritmo antes (ou no lugar) de escrevê-lo em uma linguagem de programação:

- **Descrição narrativa:** o algoritmo escrito em linguagem natural, como fizemos no Capítulo 1.
- **Fluxograma:** representação gráfica utilizando símbolos padronizados (retângulos, losangos, setas etc.).
- **Pseudocódigo (Portugol):** uma linguagem intermediária entre o português e uma linguagem de programação real, com uma sintaxe estruturada. É o que utilizaremos no VisuAlg.

### 2.3 Símbolos básicos de fluxograma

| Símbolo       | Nome          | Significado                                                             |
|---------------|---------------|-------------------------------------------------------------------------|
| Oval          | Terminador    | Indica o início ou o fim do algoritmo                                   |
| Paralelogramo | Entrada/Saída | Representa a leitura de dados ou a exibição de resultados               |
| Retângulo     | Processamento | Representa uma ação, cálculo ou atribuição                              |
| Losango       | Decisão       | Representa uma condição, com dois caminhos possíveis (verdadeiro/falso) |
| Seta          | Fluxo         | Indica a direção e a sequência da execução                              |



```

1 Algoritmo "CalcularSalarioProfessor"
2 // Data atual : 04/09/2026
3 Var
4 // Seção de Declarações das variáveis
5 HT : Real // Horas trabalhadas no mês
6 VH : Real // Valor hora-aula
7 PD : Real // Percentual de desconto
8 SB : Real // Salário bruto
9 TD : Real // Total de desconto
10 SL : Real // Salário líquido
11
12 Inicio
13 // Seção de Comandos, procedimento, funções, operadores, etc...
14 // 1. Estabelecer a leitura da variável HT
15 Escreva("Digite a quantidade de horas trabalhadas no mês (HT): ")
16 Leia(HT)
17
18 // 2. Estabelecer a leitura da variável VH
19 Escreva("Digite o valor da hora-aula (VH): ")
20 Leia(VH)
21
22 // 3. Estabelecer a leitura da variável PD
23 Escreva("Digite o percentual de desconto do INSS (PD): ")
24 Leia(PD)
25
26 // 4. Calcular o salário bruto (SB)
27 SB <- HT * VH
28
29 // 5. Calcular o total de desconto (TD)
30 TD <- (PD / 100) * SB
31
32 // 6. Calcular o salário líquido (SL)
33 SL <- SB - TD
34
35 // 7. Apresentar os valores dos salários bruto e líquido
36 Escreva("Salário Bruto (SB): R$ ", SB:1:2)
37 Escreva("Salário Líquido (SL): R$ ", SL:1:2)
38
39 Fimalgoritmo
  
```

## 2.4 Exemplo — do problema ao algoritmo

Problema: calcular a média de um aluno a partir de duas notas e informar se ele foi aprovado (média maior ou igual a 6).

```

Início
1. Ler a primeira nota (nota1)
2. Ler a segunda nota (nota2)
3. Calcular a média: media <- (nota1 + nota2) / 2
4. Se media >= 6, então escrever "Aprovado"
   Senão escrever "Reprovado"
Fim
  
```

## 2.5 Exercícios — Capítulo 2

1. Descreva, em passos numerados, um algoritmo para verificar se um número digitado pelo usuário é par ou ímpar.
2. Descreva um algoritmo para calcular a área de um retângulo, sabendo que o usuário informará a base e a altura.
3. Explique, com suas palavras, o que representa o símbolo de losango em um fluxograma.

4. Escreva o algoritmo (em passos numerados) para determinar o maior entre três números informados pelo usuário.

## Capítulo 3 — Conhecendo o VisuAlg e o Portugol

### 3.1 O que é o Portugol?

Portugol (contração de "Português" + "Algol") é uma pseudolinguagem de programação estruturada, escrita com comandos em português. Ela não serve para criar programas comerciais, mas é uma excelente ferramenta didática, pois permite que o aluno aprenda a lógica de programação sem a barreira de uma língua estrangeira (como o inglês, usado em linguagens como C, Java ou Python).

### 3.2 O que é o VisuAlg?

VisuAlg é um ambiente de desenvolvimento (editor e interpretador) gratuito, muito utilizado no ensino de lógica de programação no Brasil, que permite escrever, executar e depurar algoritmos em Portugol. Ele mostra o resultado da execução do algoritmo passo a passo, o que facilita a compreensão de como cada instrução altera os valores das variáveis.

### 3.3 Estrutura básica de um algoritmo em VisuAlg

Todo algoritmo escrito no VisuAlg segue uma estrutura fixa, composta por três partes principais:

```
algoritmo "Nome do Algoritmo"
// Seção de declaração de variáveis
var
    nome_variavel: tipo

inicio
    // Seção de comandos
    // (o que o algoritmo efetivamente faz)
finalgoritmo
```

#### Explicando cada parte

- algoritmo "...": define o nome do programa (obrigatório, sempre entre aspas).
- var: palavra reservada que inicia a seção onde declaramos todas as variáveis que serão usadas.
- inicio: marca o começo do bloco de comandos que serão executados.
- finalgoritmo: marca o fim do programa.
- // texto: cria um comentário — um texto explicativo que o VisuAlg ignora ao executar o programa.

### 3.4 Primeiro programa: "Olá, Mundo!"

É tradição, ao aprender qualquer linguagem de programação, começar com um programa simples que exibe uma mensagem na tela. Veja como isso é feito em VisuAlg:

```
algoritmo "Ola Mundo"
inicio
    escreva("Ola, Mundo!")
finalgoritmo
```

O comando `escreva()` é usado para exibir textos ou valores na tela. Tudo o que estiver entre aspas dentro dos parênteses será mostrado exatamente como está escrito.

**Atenção**

O VisuAlg pode não reconhecer corretamente acentos e caracteres especiais (ç, ã, é, ê etc.) em todas as versões e configurações. Por isso, é uma boa prática evitar acentuação dentro dos textos e comentários dos algoritmos, escrevendo, por exemplo, "Ola" em vez de "Olá".

### 3.5 Regras gerais de sintaxe

- O VisuAlg não diferencia maiúsculas de minúsculas (não é case sensitive) para palavras reservadas.
- Cada comando é normalmente escrito em uma linha própria.
- Nomes de variáveis não podem ter espaços, acentos ou começar com número.
- Toda string (texto) deve estar entre aspas duplas.
- O símbolo `<-` é usado para atribuir um valor a uma variável (equivale ao sinal de igual em outras linguagens).

### 3.6 Exercícios — Capítulo 3

1. Escreva, no VisuAlg, um algoritmo que exiba na tela seu nome completo e o curso que você está fazendo.
2. Explique com suas palavras a função das palavras reservadas `algoritmo`, `var`, `inicio` e `fimalgoritmo`.
3. Escreva um algoritmo que exiba, em linhas separadas, três frases diferentes usando o comando `escreva()`.
4. Pesquise e explique para que serve o comando `escreval()` no VisuAlg, e qual a diferença dele para o comando `escreva()`.

## Capítulo 4 — Variáveis, Constantes e Tipos de Dados

### 4.1 O que são variáveis?

Variáveis são espaços de memória utilizados para armazenar dados que podem ser alterados durante a execução do programa. Cada variável possui um nome (identificador) e um tipo de dado associado, que define que tipo de informação ela pode armazenar.

### 4.2 Regras para nomear variáveis

- Devem começar com uma letra (nunca com número).
- Não podem conter espaços, acentos ou caracteres especiais (use \_ para separar palavras, como `media_final`).
- Não podem ser iguais a palavras reservadas da linguagem (como `inicio`, `var`, `se`, etc.).
- Devem ter nomes que representem claramente a informação armazenada (`idade`, `nome_aluno`, `salario`).

### 4.3 Tipos de dados primitivos no VisuAlg

| Tipo      | Descrição                                                    | Exemplo de valor     |
|-----------|--------------------------------------------------------------|----------------------|
| inteiro   | Números inteiros, positivos ou negativos, sem casas decimais | 10, -5, 0, 200       |
| real      | Números com casas decimais (ponto flutuante)                 | 3.14, -0.5, 10.0     |
| caractere | Texto, cadeia de caracteres (string)                         | "Amazonas", "Manaus" |
| logico    | Valor booleano: apenas verdadeiro ou falso                   | verdadeiro, falso    |

### 4.4 Declarando variáveis

As variáveis são declaradas na seção `var`, antes do início do bloco de comandos. Podemos declarar várias variáveis do mesmo tipo na mesma linha, separadas por vírgula:

```
algoritmo "Declaracao de Variaveis"
var
    nome: caractere
    idade: inteiro
    altura: real
    aprovado: logico
    nota1, nota2, media: real

inicio
    // comandos aqui
fimalgoritmo
```

### 4.5 Constantes

Constantes são valores que não se alteram durante a execução do programa. No VisuAlg, elas são declaradas com a palavra reservada `const`, geralmente antes da seção `var`:

```

algoritmo "Uso de Constante"
const
  PI = 3.14159
var
  raio, area: real

inicio
  escreva("Digite o raio: ")
  leia(raio)
  area <- PI * raio * raio
  escreva("A area do circulo e: ", area)
fimalgoritmo

```

## 4.6 Atribuição de valores

A atribuição é feita com o operador <- (seta para a esquerda), que armazena um valor (ou resultado de uma expressão) dentro de uma variável:

```

idade <- 18
nome <- "Priscila"
media <- (nota1 + nota2) / 2
aprovado <- verdadeiro

```

### Dica da professora

Pense na variável como uma caixa com uma etiqueta (o nome) que guarda um valor (o conteúdo). Esse conteúdo pode ser trocado a qualquer momento, mas a caixa sempre guarda apenas um valor de cada vez, do tipo definido na declaração.

## 4.7 Exercícios — Capítulo 4

1. Declare variáveis adequadas (com nome e tipo corretos) para armazenar: o nome de um produto, seu preço, a quantidade em estoque e se ele está em promoção.
2. Explique a diferença entre uma variável do tipo inteiro e uma do tipo real, dando um exemplo de uso para cada uma.
3. Escreva um algoritmo que declare uma constante para o valor do salário mínimo e uma variável para calcular o dobro desse valor.
4. Aponte o que está incorreto nos seguintes nomes de variáveis e reescreva-os corretamente: "2idade", "nome completo", "salário", "var".

## Capítulo 5 — Operadores e Expressões

### 5.1 Operadores aritméticos

Usados para realizar cálculos matemáticos entre valores numéricos:

| Operador | Operação                  | Exemplo                         |
|----------|---------------------------|---------------------------------|
| +        | Adição                    | $5 + 3$ resulta em 8            |
| -        | Subtração                 | $5 - 3$ resulta em 2            |
| *        | Multiplicação             | $5 * 3$ resulta em 15           |
| /        | Divisão (sempre real)     | $5 / 2$ resulta em 2.5          |
| div      | Divisão inteira           | $5 \text{ div } 2$ resulta em 2 |
| mod      | Resto da divisão (módulo) | $5 \text{ mod } 2$ resulta em 1 |
| ^        | Potenciação               | $2 ^ 3$ resulta em 8            |

### 5.2 Operadores relacionais

Usados para comparar dois valores, sempre retornando verdadeiro ou falso:

| Operador | Significado      | Exemplo                         |
|----------|------------------|---------------------------------|
| =        | Igual a          | $5 = 5 \rightarrow$ verdadeiro  |
| <>       | Diferente de     | $5 <> 3 \rightarrow$ verdadeiro |
| >        | Maior que        | $5 > 3 \rightarrow$ verdadeiro  |
| <        | Menor que        | $5 < 3 \rightarrow$ falso       |
| >=       | Maior ou igual a | $5 >= 5 \rightarrow$ verdadeiro |
| <=       | Menor ou igual a | $5 <= 3 \rightarrow$ falso      |

### 5.3 Operadores lógicos

Usados para combinar duas ou mais condições (expressões lógicas):

| Operador | Significado                                               | Exemplo                                                                    |
|----------|-----------------------------------------------------------|----------------------------------------------------------------------------|
| e        | Verdadeiro somente se as duas condições forem verdadeiras | $(idade \geq 18) \text{ e } (possui\_cnh = \text{verdadeiro})$             |
| ou       | Verdadeiro se pelo menos uma condição for verdadeira      | $(chove = \text{verdadeiro}) \text{ ou } (esta\_frio = \text{verdadeiro})$ |
| nao      | Inverte o valor lógico da condição                        | nao (aprovado)                                                             |

### 5.4 Ordem de precedência dos operadores

Assim como na matemática, os operadores possuem uma ordem de prioridade na hora de serem calculados. Do maior para o menor grau de prioridade:

6. Parênteses ( )
7. Potenciação (^)
8. Multiplicação, divisão, div e mod (\*, /, div, mod)
9. Adição e subtração (+, -)
10. Operadores relacionais (=, <>, >, <, >=, <=)
11. Operador lógico nao
12. Operador lógico e
13. Operador lógico ou

#### Dica da professora

Na dúvida sobre a ordem em que uma expressão será calculada, use parênteses! Eles deixam o código mais claro e evitam erros de lógica, além de ajudarem outras pessoas a entenderem seu raciocínio.

## 5.5 Exemplo prático

```
algoritmo "Operadores"
var
    a, b, resultado: inteiro

inicio
    a <- 10
    b <- 3
    resultado <- (a + b) * 2
    escreva("Resultado: ", resultado)
    escreva("Divisao inteira: ", a div b)
    escreva("Resto: ", a mod b)
fimalgoritmo
```

## 5.6 Exercícios — Capítulo 5

1. Calcule manualmente o resultado das expressões:  $2 + 3 * 4$ ;  $(2 + 3) * 4$ ;  $10 \text{ div } 3$ ;  $10 \text{ mod } 3$ .
2. Escreva a expressão lógica em Portugol que representa: "a pessoa pode votar se tiver 16 anos ou mais".
3. Escreva um algoritmo que leia dois números inteiros e exiba a soma, a subtração, o produto, a divisão real e o resto da divisão entre eles.
4. Escreva a expressão lógica que verifica se um número é, ao mesmo tempo, maior que 0 e menor que 100.

## Capítulo 6 — Entrada e Saída de Dados

### 6.1 Comandos de saída: escreva() e escreval()

Ambos exibem dados na tela. A diferença é que `escreval()` pula uma linha após exibir o conteúdo, enquanto `escreva()` mantém o cursor na mesma linha:

```
algoritmo "Comandos de Saida"
inicio
    escreva("Isto fica ")
    escreva("na mesma linha")
    escreval("")
    escreval("Isto fica em uma linha")
    escreval("E isto em outra linha")
fimalgoritmo
```

### 6.2 Comando de entrada: leia()

O comando `leia()` interrompe a execução do algoritmo e aguarda que o usuário digite um valor, que será armazenado na variável indicada entre parênteses. É fundamental que a variável já tenha sido declarada com o tipo compatível com o dado que será digitado.

```
algoritmo "Entrada de Dados"
var
    nome: caractere
    idade: inteiro

inicio
    escreva("Digite seu nome: ")
    leia(nome)
    escreva("Digite sua idade: ")
    leia(idade)
    escreval("Ola, ", nome, "! Voce tem ", idade, " anos.")
fimalgoritmo
```

### 6.3 Concatenando textos e variáveis

É possível exibir, em um único comando `escreva()` ou `escreval()`, textos fixos e conteúdos de variáveis, separando cada elemento por vírgula:

```
escreval("O aluno ", nome, " tem
", idade, " anos e mora em
Manaus.")
```

#### Atenção

O comando `leia()` espera um valor compatível com o tipo da variável. Se a variável for do tipo inteiro e o usuário digitar um texto, o programa apresentará erro. Sempre planeje os tipos de dados de acordo com a informação que será realmente digitada.

## 6.4 Exemplo completo — Calculadora de Média

```
algoritmo "Calculadora de Media"
var
    nome: caractere
    nota1, nota2, media: real

inicio
    escreva("Digite o nome do aluno: ")
    leia(nome)
    escreva("Digite a primeira nota: ")
    leia(nota1)
    escreva("Digite a segunda nota: ")
    leia(nota2)

    media <- (nota1 + nota2) / 2

    escreval("Aluno: ", nome)
    escreval("Media final: ", media)
fimalgoritmo
```

## 6.5 Exercícios — Capítulo 6

1. Escreva um algoritmo que leia o nome e o preço de um produto e exiba uma mensagem informando os dois valores.
2. Escreva um algoritmo que leia dois números e exiba a soma entre eles, usando escreval().
3. Escreva um algoritmo que leia a base e a altura de um triângulo e calcule sua área ( $\text{base} * \text{altura} / 2$ ).
4. Explique, com suas palavras, a diferença entre os comandos escreva() e escreval().

## Capítulo 7 — Estruturas Condicionais

### 7.1 Por que usar estruturas condicionais?

As estruturas condicionais (também chamadas de estruturas de seleção ou de decisão) permitem que o programa tome decisões, executando diferentes blocos de comando dependendo se uma condição é verdadeira ou falsa. Elas são essenciais para criar algoritmos que reagem a diferentes situações.

### 7.2 Seleção simples: se ... entao

```
se (condicao) entao
    // comandos executados se a condicao for verdadeira
fimse
```

Exemplo: verificar se um número é positivo.

```
algoritmo "Numero Positivo"
var
    numero: inteiro

inicio
    escreva("Digite um numero: ")
    leia(numero)

    se (numero > 0) entao
        escreval("O numero e positivo.")
    fimse
fimalgoritmo
```

### 7.3 Seleção composta: se ... entao ... senao

```
se (condicao) entao
    // comandos se a condicao for verdadeira
senao
    // comandos se a condicao for falsa
fimse
```

Exemplo: verificar aprovação de um aluno.

```
algoritmo "Aprovacao"
var
    media: real

inicio
    escreva("Digite a media final: ")
    leia(media)

    se (media >= 6) entao
        escreval("Aluno aprovado!")
    senao
        escreval("Aluno reprovado.")
    fimse
fimalgoritmo
```

## 7.4 Condições encadeadas: senao se

Quando existem mais de duas possibilidades, podemos encadear várias condições usando senao se:

```
algoritmo "Classificacao por Faixa Etaria"
var
    idade: inteiro

inicio
    escreva("Digite a idade: ")
    leia(idade)

    se (idade < 12) entao
        escreval("Crianca")
    senao se (idade < 18) entao
        escreval("Adolescente")
    senao se (idade < 60) entao
        escreval("Adulto")
    senao
        escreval("Idoso")
    fimse
fimalgoritmo
```

## 7.5 Seleção múltipla: escolha ... caso

Quando uma mesma variável precisa ser comparada com vários valores possíveis (por exemplo, um menu de opções), a estrutura escolha (também chamada de caso) deixa o código mais organizado do que vários se encadeados:

```
algoritmo "Menu de Opcoes"
var
    opcao: inteiro

inicio
    escreval("1 - Cadastrar")
    escreval("2 - Consultar")
    escreval("3 - Excluir")
    escreva("Escolha uma opcao: ")
    leia(opcao)

    escolha opcao
        caso 1
            escreval("Voce escolheu Cadastrar")
        caso 2
            escreval("Voce escolheu Consultar")
        caso 3
            escreval("Voce escolheu Excluir")
        outrocaso
            escreval("Opcao invalida")
    fimescolha
fimalgoritmo
```

**Dica da professora**

Use escolha...caso quando estiver comparando UMA mesma variável com vários valores fixos (como em um menu). Quando as condições envolverem faixas de valores ou comparações diferentes, prefira se...senao se.

## 7.6 Exercícios — Capítulo 7

1. Escreva um algoritmo que leia um número e informe se ele é par ou ímpar (dica: use o operador mod).
2. Escreva um algoritmo que leia três notas de um aluno, calcule a média e classifique o resultado em: "Aprovado" (média  $\geq 7$ ), "Recuperação" (média entre 5 e 6.9) ou "Reprovado" (média menor que 5).
3. Escreva um algoritmo, usando escolha...caso, que leia o número de um mês (1 a 12) e exiba o nome da estação do ano correspondente (considere o hemisfério sul).
4. Escreva um algoritmo que leia três números e exiba o maior deles, utilizando estruturas condicionais.

## Capítulo 8 — Estruturas de Repetição

### 8.1 Por que usar estruturas de repetição?

As estruturas de repetição (também chamadas de laços ou loops) permitem executar um mesmo bloco de comandos várias vezes, sem precisar reescrevê-lo. Elas são fundamentais quando não sabemos, de antemão, quantas vezes uma ação precisará ser repetida, ou quando queremos repetir uma tarefa um número exato de vezes.

### 8.2 Estrutura para (contador definido)

Usada quando sabemos exatamente quantas vezes o bloco de comandos deve se repetir:

```
para variavel de inicio ate fim faca
    // comandos repetidos
fimpara
```

Exemplo: exibir os números de 1 a 10.

```
algoritmo "Contagem"
var
    i: inteiro

inicio
    para i de 1 ate 10 faca
        escreval(i)
    fimpara
fimalgoritmo
```

Também é possível definir o incremento (passo) usando a cláusula passo:

```
para i de 10 ate 1 passo -1 faca
    escreval(i)
fimpara
```

### 8.3 Estrutura enquanto (repetição com teste no início)

O bloco de comandos é repetido enquanto a condição for verdadeira. A condição é testada antes de cada repetição — por isso, se a condição já começar falsa, o bloco pode nunca ser executado.

```
enquanto (condicao) faca
    // comandos repetidos
fimenquanto
```

Exemplo: somar números digitados pelo usuário até que ele digite 0.

```
algoritmo "Soma ate Zero"
var
    numero, soma: inteiro

inicio
```

```

soma <- 0
escreva("Digite um numero (0 para parar): ")
leia(numero)

enquanto (numero <> 0) faca
    soma <- soma + numero
    escreva("Digite um numero (0 para parar): ")
    leia(numero)
fimenquanto

escreval("A soma total foi: ", soma)
fimalgoritmo

```

## 8.4 Estrutura repita ... ate (repetição com teste no final)

Nesta estrutura, o bloco de comandos é executado ao menos uma vez, pois a condição é verificada somente depois de cada repetição. O laço se repete até que a condição se torne verdadeira (é o oposto lógico do enquanto).

```

repita
    // comandos repetidos
ate (condicao)

```

Exemplo: pedir uma senha até que o usuário acerte.

```

algoritmo "Validar Senha"
var
    senha: inteiro

inicio
    repita
        escreva("Digite a senha (1234): ")
        leia(senha)
    ate (senha = 1234)

    escreval("Senha correta! Acesso liberado.")
fimalgoritmo

```

## 8.5 Comparando as três estruturas

| Estrutura    | Quando usar                                                      | Teste da condição                        |
|--------------|------------------------------------------------------------------|------------------------------------------|
| para         | Número de repetições conhecido/definido                          | Controlado automaticamente pelo contador |
| enquanto     | Número de repetições desconhecido; pode não executar nenhuma vez | No início do laço                        |
| repita...ate | Número de repetições desconhecido; executa ao menos uma vez      | No final do laço                         |

**Atenção**

Cuidado com laços infinitos! Isso acontece quando a condição de parada nunca se torna falsa (no enquanto) ou nunca se torna verdadeira (no repita...ate), geralmente porque o programador esquece de atualizar, dentro do laço, a variável usada na condição.

## 8.6 Exercícios — Capítulo 8

1. Escreva um algoritmo que exiba, usando a estrutura para, todos os números pares entre 1 e 50.
2. Escreva um algoritmo que calcule o fatorial de um número digitado pelo usuário (dica: fatorial de 5 é  $5*4*3*2*1$ ).
3. Escreva um algoritmo que leia números digitados pelo usuário e pare quando for digitado um número negativo, exibindo ao final a soma e a média dos números positivos digitados.
4. Escreva um algoritmo, usando repita...ate, que simule um caixa eletrônico pedindo repetidamente uma senha de 4 dígitos até que o usuário acerte, permitindo no máximo 3 tentativas.

## Capítulo 9 — Vetores (Introdução)

### 9.1 O que é um vetor?

Um vetor (também chamado de array ou variável composta homogênea) é uma estrutura que permite armazenar vários valores do mesmo tipo em uma única variável, organizados em posições numeradas (índices). Isso evita a necessidade de criar uma variável separada para cada valor.

### 9.2 Declarando um vetor

```
var
    notas: vetor[1..5] de real
```

O trecho acima declara um vetor chamado notas, capaz de armazenar 5 valores reais, nas posições de 1 a 5.

### 9.3 Acessando e preenchendo um vetor

```
algoritmo "Vetor de Notas"
var
    notas: vetor[1..5] de real
    i: inteiro
    soma, media: real

inicio
    soma <- 0

    para i de 1 ate 5 faca
        escreva("Digite a nota ", i, ": ")
        leia(notas[i])
        soma <- soma + notas[i]
    fimpara

    media <- soma / 5
    escreval("A media da turma e: ", media)
finalgoritmo
```

#### Dica da professora

Cada posição de um vetor é acessada pelo seu índice, escrito entre colchetes, como em notas[1] ou notas[i]. Vetores são muito úteis quando combinados com a estrutura para, pois permitem percorrer todas as posições automaticamente.

## **9.4 Exercícios — Capítulo 9**

1. Escreva um algoritmo que leia 10 números inteiros em um vetor e exiba o maior valor digitado.
2. Escreva um algoritmo que leia um vetor de 5 nomes (caractere) e depois exiba todos eles em ordem.
3. Escreva um algoritmo que leia 5 números em um vetor e informe quantos deles são pares.

## Capítulo 10 — Projeto Prático Final

### 10.1 Objetivo do projeto

Este capítulo reúne, em um único projeto, todos os conceitos estudados na apostila: variáveis, tipos de dados, operadores, entrada e saída, estruturas condicionais, estruturas de repetição e vetores. O objetivo é que o aluno pratique a construção de um algoritmo completo, aplicando o raciocínio lógico do início ao fim.

### 10.2 Proposta: Sistema de Cadastro de Produtos

Desenvolva, em VisuAlg, um algoritmo que simule um pequeno sistema de cadastro de produtos para uma loja, contendo um menu com as seguintes opções:

14. Cadastrar produto (nome e preço)
15. Listar todos os produtos cadastrados
16. Calcular o valor total do estoque (soma de todos os preços)
17. Sair do sistema

O programa deve utilizar:

- Um vetor para armazenar os nomes dos produtos e outro para os preços.
- A estrutura escolha...caso para o menu de opções.
- A estrutura repita...ate (ou enquanto) para manter o menu em execução até que o usuário escolha sair.
- A estrutura para para percorrer os vetores ao listar produtos ou calcular o total.

### 10.3 Estrutura sugerida do algoritmo

```
algoritmo "Cadastro de Produtos"
const
  MAX = 10
var
  nomes: vetor[1..MAX] de caractere
  precos: vetor[1..MAX] de real
  quantidade, opcao, i: inteiro
  total: real

inicio
  quantidade <- 0

  repita
    escreval("===== MENU =====")
    escreval("1 - Cadastrar produto")
    escreval("2 - Listar produtos")
    escreval("3 - Valor total do estoque")
    escreval("4 - Sair")
    escreva("Escolha uma opcao: ")
    leia(opcao)

    escolha opcao
      caso 1
```

```

    se (quantidade < MAX) entao
        quantidade <- quantidade + 1
        escreva("Nome do produto: ")
        leia(nomes[quantidade])
        escreva("Preco do produto: ")
        leia(precos[quantidade])
    senao
        escreval("Estoque cheio!")
    fimse

caso 2
    para i de 1 ate quantidade faca
        escreval(nomes[i], " - R$ ", precos[i])
    fimpara

caso 3
    total <- 0
    para i de 1 ate quantidade faca
        total <- total + precos[i]
    fimpara
    escreval("Valor total do estoque: R$ ", total)

caso 4
    escreval("Encerrando o sistema...")

outrocaso
    escreval("Opcao invalida!")
fimescolha

ate (opcao = 4)
fimalgoritmo

```

## 10.4 Desafios extras

1. Adicione ao sistema uma opção para buscar um produto pelo nome e exibir seu preço.
2. Adicione uma opção para calcular e exibir o preço médio dos produtos cadastrados.
3. Modifique o sistema para não permitir o cadastro de produtos com preço negativo, exibindo uma mensagem de erro nesse caso.
4. Adicione uma opção que exiba o produto mais caro e o mais barato cadastrados.

## **Referências**

- FORBELLONE, André Luiz Villar; EBERSPÄCHER, Henri Frederico. *Lógica de Programação: a construção de algoritmos e estruturas de dados*. São Paulo: Pearson, 2005.
- MANZANO, José Augusto N. G.; OLIVEIRA, Jayr Figueiredo de. *Algoritmos: lógica para desenvolvimento de programação de computadores*. São Paulo: Érica, 2019.
- ASCENCIO, Ana Fernanda Gomes; CAMPOS, Edilene Aparecida Veneruchi de. *Fundamentos da Programação de Computadores: algoritmos, Pascal, C/C++ e Java*. São Paulo: Pearson, 2012.
- VisuAlg — Ambiente de desenvolvimento de algoritmos em Portugol. Material de apoio elaborado pela Profa. Priscila Gonçalves para o CETAM.